

```
In [1]: import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

```
In [2]: import os
print(os.listdir())
```

```
Out:
['.ipynb_checkpoints', 'Salary Prediction of Data Professions.csv', 'Untitled.ipynb']
```

```
In [3]: import pandas as pd

df = pd.read_csv("Salary Prediction of Data Professions.csv")

df.head()
```

Out:

	FIRST NAME	LAST NAME	SEX	DOJ	CURRENT DATE	DESIGNATION	AGE	SALARY	UNIT	LEAVES USED	LEAVES REMAINING	RATINGS	PAST EXP
0	TOMASA	ARMEN	F	5-18-2014	01-07-2016	Analyst	21.0	44570	Finance	24.0	6.0	2.0	0
1	ANNIE	NaN	F	NaN	01-07-2016	Associate	NaN	89207	Web	NaN	13.0	NaN	7
2	OLIVE	ANCY	F	7-28-2014	01-07-2016	Analyst	21.0	40955	Finance	23.0	7.0	3.0	0
3	CHERRY	AQUILAR	F	04-03-2013	01-07-2016	Analyst	22.0	45550	IT	22.0	8.0	3.0	0
4	LEON	ABOULAHOU	M	11-20-2014	01-07-2016	Analyst	NaN	43161	Operations	27.0	3.0	NaN	3

```
In [4]: print(df.columns)
```

```
Out:
Index(['FIRST NAME', 'LAST NAME', 'SEX', 'DOJ', 'CURRENT DATE', 'DESIGNATION',
      'AGE', 'SALARY', 'UNIT', 'LEAVES USED', 'LEAVES REMAINING', 'RATINGS',
      'PAST EXP'],
      dtype='object')
```

```
In [5]: print(df.columns)
```

Out:

```
Index(['FIRST NAME', 'LAST NAME', 'SEX', 'DOJ', 'CURRENT DATE', 'DESIGNATION',
      'AGE', 'SALARY', 'UNIT', 'LEAVES USED', 'LEAVES REMAINING', 'RATINGS',
      'PAST EXP'],
      dtype='object')
```

In [6]: df.head()

Out:

	FIRST NAME	LAST NAME	SEX	DOJ	CURRENT DATE	DESIGNATION	AGE	SALARY	UNIT	LEAVES USED	LEAVES REMAINING	RATINGS	PAST EXP
0	TOMASA	ARMEN	F	5-18-2014	01-07-2016	Analyst	21.0	44570	Finance	24.0	6.0	2.0	0
1	ANNIE	NaN	F	NaN	01-07-2016	Associate	NaN	89207	Web	NaN	13.0	NaN	7
2	OLIVE	ANCY	F	7-28-2014	01-07-2016	Analyst	21.0	40955	Finance	23.0	7.0	3.0	0
3	CHERRY	AQUILAR	F	04-03-2013	01-07-2016	Analyst	22.0	45550	IT	22.0	8.0	3.0	0
4	LEON	ABOULAHOU	M	11-20-2014	01-07-2016	Analyst	NaN	43161	Operations	27.0	3.0	NaN	3

In [7]: print(df.shape)

```
print(df.columns)
```

```
df.info()
```

```
df.describe()
```

Out:

```
(2639, 13)
Index(['FIRST NAME', 'LAST NAME', 'SEX', 'DOJ', 'CURRENT DATE', 'DESIGNATION',
      'AGE', 'SALARY', 'UNIT', 'LEAVES USED', 'LEAVES REMAINING', 'RATINGS',
      'PAST EXP'],
      dtype='object')
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2639 entries, 0 to 2638
Data columns (total 13 columns):
#   Column          Non-Null Count  Dtype
---  -
0   FIRST NAME      2639 non-null  object
1   LAST NAME       2637 non-null  object
2   SEX             2639 non-null  object
3   DOJ             2638 non-null  object
4   CURRENT DATE    2639 non-null  object
5   DESIGNATION     2639 non-null  object
6   AGE             2636 non-null  float64
7   SALARY          2639 non-null  int64
8   UNIT            2639 non-null  object
9   LEAVES USED     2636 non-null  float64
10  LEAVES REMAINING 2637 non-null  float64
11  RATINGS         2637 non-null  float64
12  PAST EXP        2639 non-null  int64
```

dtypes: float64(4), int64(2), object(7)
memory usage: 268.2+ KB

	AGE	SALARY	LEAVES USED	LEAVES REMAINING	RATINGS	PAST EXP
count	2636.000000	2639.000000	2636.000000	2637.000000	2637.000000	2639.000000
mean	24.756449	58136.678287	22.501517	7.503223	3.486159	1.566881
std	3.908228	36876.956944	4.604469	4.603193	1.114933	2.728416
min	21.000000	40001.000000	15.000000	0.000000	2.000000	0.000000
25%	22.000000	43418.000000	19.000000	4.000000	2.000000	0.000000
50%	24.000000	46781.000000	22.000000	8.000000	3.000000	1.000000
75%	25.000000	51401.500000	26.000000	11.000000	4.000000	2.000000
max	45.000000	388112.000000	30.000000	15.000000	5.000000	23.000000

```
In [8]: df = df.dropna()
```

```
In [9]: print(df.columns)
```

Out:

```
Index(['FIRST NAME', 'LAST NAME', 'SEX', 'DOJ', 'CURRENT DATE', 'DESIGNATION',  
      'AGE', 'SALARY', 'UNIT', 'LEAVES USED', 'LEAVES REMAINING', 'RATINGS',  
      'PAST EXP'],  
      dtype='object')
```

```
In [10]: from sklearn.model_selection import train_test_split  
  
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.2,  
    random_state=42  
)
```

Out:

```
-----  
NameError                                Traceback (most recent call last)  
Cell In[15], line 4  
      1 from sklearn.model_selection import train_test_split  
      3 X_train, X_test, y_train, y_test = train_test_split(  
----> 4     X,  
      5     y,  
      6     test_size=0.2,  
      7     random_state=42  
      8 )  
  
NameError: name 'X' is not defined
```

```
In [11]: X = df.drop('Salary', axis=1)  
        y = df['Salary']
```

Out:

```
-----  
KeyError                                Traceback (most recent call last)
```

```

Cell In[16], line 1
----> 1 X = df.drop('Salary', axis=1)
      2 y = df['Salary']

File ~\anaconda2\Lib\site-packages\pandas\core\frame.py:5603, in DataFrame.drop(self, labels, axis, index, columns, level, inplace, errors)
    5455 def drop(
    5456     self,
    5457     labels: IndexLabel | None = None,
    (...) 5464     errors: IgnoreRaise = "raise",
    5465 ) -> DataFrame | None:
    5466     """
    5467     Drop specified labels from rows or columns.
    5468
    (...) 5601         weight 1.0      0.8
    5602     """
-> 5603     return super().drop(
    5604         labels=labels,
    5605         axis=axis,
    5606         index=index,
    5607         columns=columns,
    5608         level=level,
    5609         inplace=inplace,
    5610         errors=errors,
    5611     )

File ~\anaconda2\Lib\site-packages\pandas\core\generic.py:4810, in NDFrame.drop(self, labels, axis, index, columns, level, inplace, errors)
    4808 for axis, labels in axes.items():
    4809     if labels is not None:
-> 4810         obj = obj._drop_axis(labels, axis, level=level, errors=errors)
    4812 if inplace:
    4813     self._update_inplace(obj)

File ~\anaconda2\Lib\site-packages\pandas\core\generic.py:4852, in NDFrame._drop_axis(self, labels, axis, level, errors, only_slice)
    4850     new_axis = axis.drop(labels, level=level, errors=errors)
    4851     else:
-> 4852     new_axis = axis.drop(labels, errors=errors)
    4853     indexer = axis.get_indexer(new_axis)
    4855 # Case for non-unique axis
    4856 else:

File ~\anaconda2\Lib\site-packages\pandas\core\indexes\base.py:7136, in Index.drop(self, labels, errors)
    7134 if mask.any():
    7135     if errors != "ignore":
-> 7136         raise KeyError(f"{labels[mask].tolist()} not found in axis")
    7137     indexer = indexer[~mask]
    7138 return self.delete(indexer)

KeyError: "['Salary'] not found in axis"

```

In [12]: `print(df.columns.tolist())`

Out:

```
['FIRST NAME', 'LAST NAME', 'SEX', 'DOJ', 'CURRENT DATE', 'DESIGNATION', 'AGE', 'SALARY', 'UNIT', 'LEAVES USED', 'LEAVES REMAINING', 'RATING S', 'PAST EXP']
```

In [13]: `print(df.columns.tolist())`

Out:

```
['FIRST NAME', 'LAST NAME', 'SEX', 'DOJ', 'CURRENT DATE', 'DESIGNATION', 'AGE', 'SALARY', 'UNIT', 'LEAVES USED', 'LEAVES REMAINING', 'RATING S', 'PAST EXP']
```

```
In [14]: X = df.drop('SALARY', axis=1)
        y = df['SALARY']
```

```
In [15]: from sklearn.preprocessing import LabelEncoder

        le = LabelEncoder()

        for col in X.columns:
            if X[col].dtype == 'object':
                X[col] = le.fit_transform(X[col].astype(str))
```

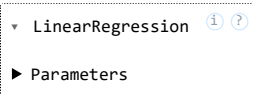
```
In [16]: from sklearn.model_selection import train_test_split

        X_train, X_test, y_train, y_test = train_test_split(
            X,
            y,
            test_size=0.2,
            random_state=42
        )
```

```
In [17]: from sklearn.linear_model import LinearRegression

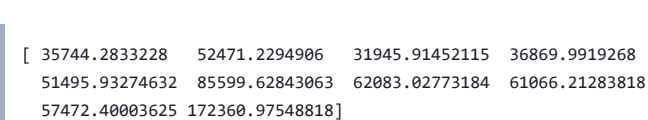
        model = LinearRegression()

        model.fit(X_train, y_train)
```

Out: 

```
In [18]: y_pred = model.predict(X_test)

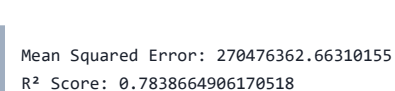
        print(y_pred[:10])
```

Out: 

```
In [19]: from sklearn.metrics import mean_squared_error, r2_score

        mse = mean_squared_error(y_test, y_pred)
        r2 = r2_score(y_test, y_pred)

        print("Mean Squared Error:", mse)
        print("R² Score:", r2)
```

Out: 

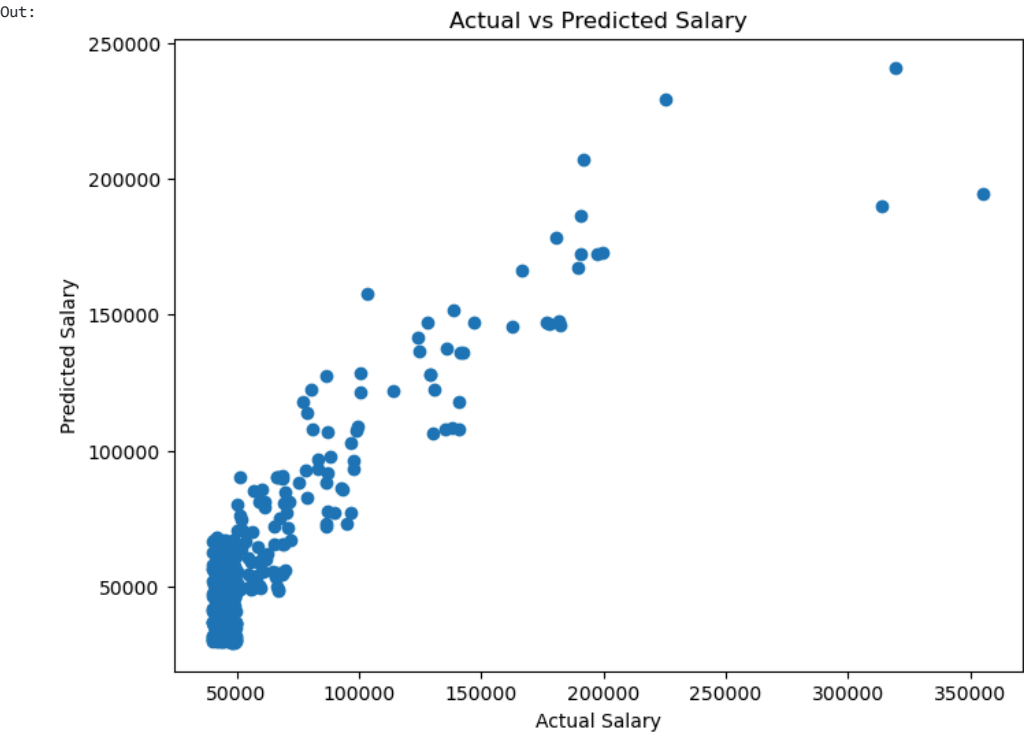
```
In [20]: import matplotlib.pyplot as plt

plt.figure(figsize=(8,6))

plt.scatter(y_test, y_pred)

plt.xlabel("Actual Salary")
plt.ylabel("Predicted Salary")
plt.title("Actual vs Predicted Salary")

plt.show()
```



```
In [21]: importance = pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': model.coef_
})

print(importance.sort_values(
    by='Coefficient',
    ascending=False
))
```

Out:

	Feature	Coefficient
6	AGE	5.300533e+03
11	PAST EXP	4.912096e+03
7	UNIT	1.118709e+02
9	LEAVES REMAINING	2.293681e+00
0	FIRST NAME	4.884299e-02
4	CURRENT DATE	-1.818989e-12
3	DOJ	-6.380804e-02
1	LAST NAME	-1.395440e+00
8	LEAVES USED	-2.293681e+00
10	RATINGS	-2.862997e+02

```
5      DESIGNATION -5.144471e+02
2      SEX -2.383072e+03
```

```
In [22]: print("Mean Squared Error:", mse)
         print("R² Score:", r2)
```

Out:

```
Mean Squared Error: 270476362.66310155
R² Score: 0.7838664906170518
```

```
In [23]: print("===== SALARY PREDICTION MODEL =====")

         print("Training Samples:", len(X_train))
         print("Testing Samples:", len(X_test))

         print("\nMean Squared Error:", mse)
         print("R² Score:", r2)
```

Out:

```
===== SALARY PREDICTION MODEL =====
Training Samples: 2104
Testing Samples: 527

Mean Squared Error: 270476362.66310155
R² Score: 0.7838664906170518
```

```
In [24]: new_employee = X.iloc[[0]]

         predicted_salary = model.predict(new_employee)

         print("Predicted Salary:", predicted_salary[0])
```

Out:

```
Predicted Salary: 30426.194980113927
```

```
In [25]: import matplotlib.pyplot as plt

         residuals = y_test - y_pred

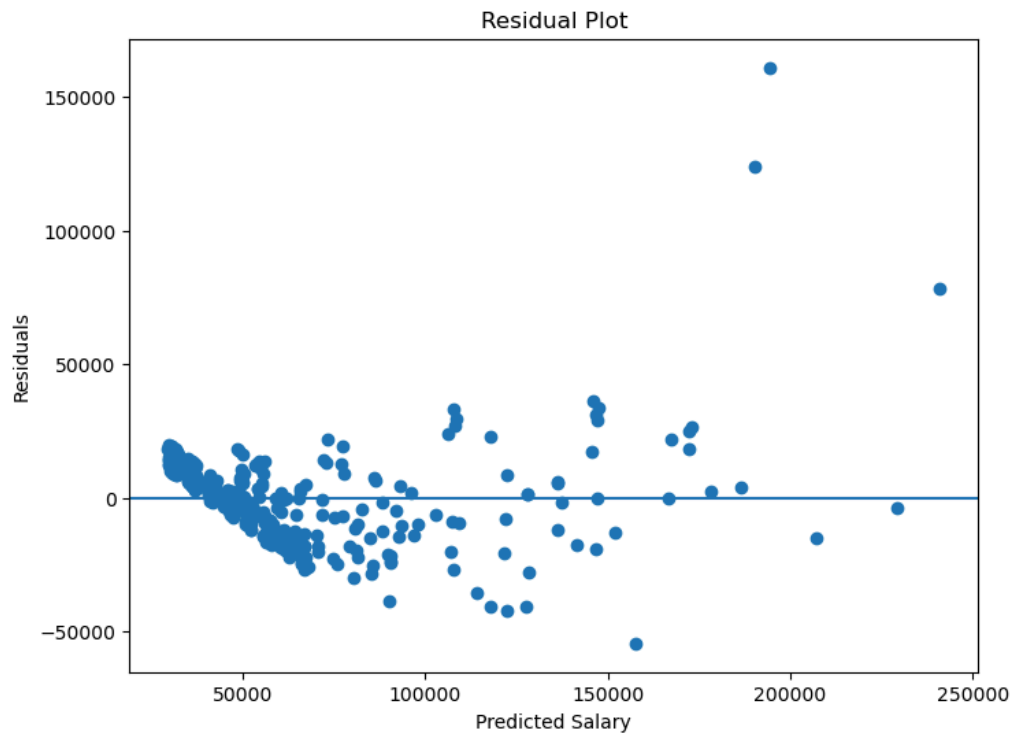
         plt.figure(figsize=(8,6))
         plt.scatter(y_pred, residuals)

         plt.axhline(y=0)

         plt.xlabel("Predicted Salary")
         plt.ylabel("Residuals")
         plt.title("Residual Plot")

         plt.show()
```

Out:



```
In [26]: import seaborn as sns

plt.figure(figsize=(12,8))

sns.heatmap(
    df.select_dtypes(include=['number']).corr(),
    annot=True,
    cmap='coolwarm'
)

plt.title("Correlation Matrix")
plt.show()
```


Out:

